SBT-DF203-Lab1_2025/FWSD/11478_ HUSSEINI SULEIMAN

HUSSEINI SULEIMAN

2025/FWSD/11478

fwsd2511478@students.icdfa.edu.ng

DATE: 7TH September, 2026

COURSE CODE: **SBT-DF203 — Basic Networking Skills for Digital Forensics**

SBT-DF203-Lab1_**2025/FWSD/11478_ HUSSEINI SULEIMAN**

LAB 1: HTTP Analysis Using Wireshark - Text Traffic

Three-Week Delivery Block 1/3 | 5-11 September 2026

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 1
Lab Title	HTTP Analysis Using Wireshark - Text Traffic
Delivery Block	1/3 of 3
Scheduled Dates	5-11 September 2026
Estimated Duration	3-4 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	basic.pcapng or basic.log generated locally; optional supplied basic.log

1. Source Materials and Slide Alignment

- HTTP, Wireshark and Digital Forensics Part 1: HTTP request/response, OSI encapsulation, TCP three-way handshake, sequence and acknowledgement numbers, connection termination and evidence reporting.
- The exercise reproduces a simple local Apache page, captures plaintext HTTP traffic, and inspects frame, IP, TCP and HTTP metadata.
- The slide assignment requires a named webpage, sender/receiver ports, initial sequence numbers, timestamps, IP addresses, MAC addresses and HTTP request timing.

Reading Requirement Before Lab Execution
The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded SBT-DF203 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

You are a junior network forensic analyst asked to document a short plaintext HTTP session. You will create a local webpage containing your name and registration number, capture the traffic, reconstruct the TCP conversation, identify the HTTP request and response, and explain how each protocol layer contributes evidence.

3. Learning Outcomes

- Explain the forensic value of HTTP, TCP, IP and Ethernet metadata.
- Create and serve a simple HTML page using Apache.
- Capture HTTP traffic using Wireshark or tshark without altering the evidence.
- Identify the SYN, SYN-ACK and ACK packets in a TCP handshake.
- Extract source/destination ports, sequence numbers, acknowledgement numbers, timestamps and HTTP headers.
- Explain why loopback captures do not show normal Ethernet MAC addresses and repeat the capture on a two-VM network where required.
- Generate a concise network forensic timeline and evidence report.

4. Legal, Ethical and Safety Rules

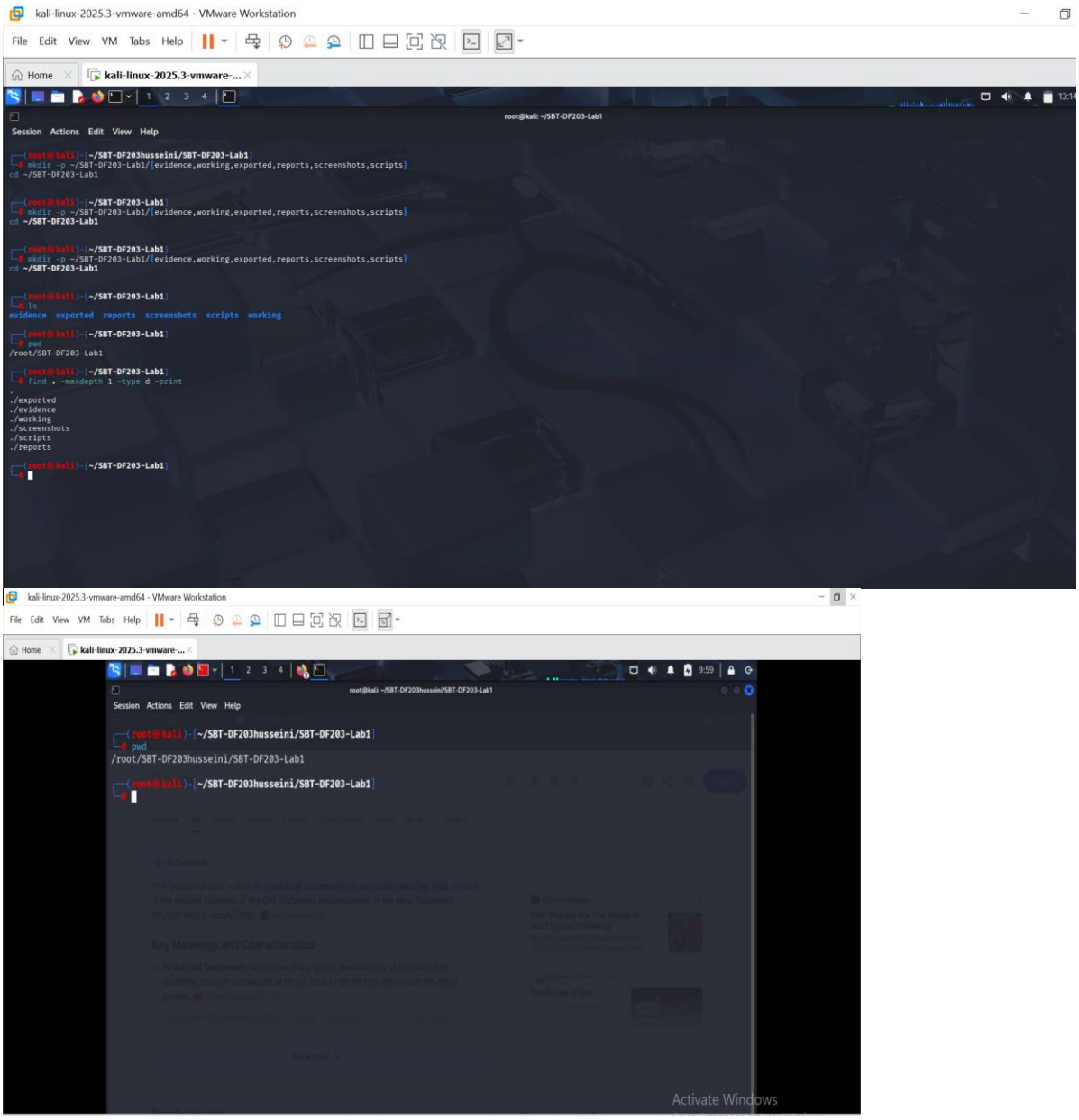
- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- Do not browse to credential-bearing or private HTTP sites for this exercise. Use only the local page or the instructor-approved NeverSSL training page.

5. Required Tools and Environment

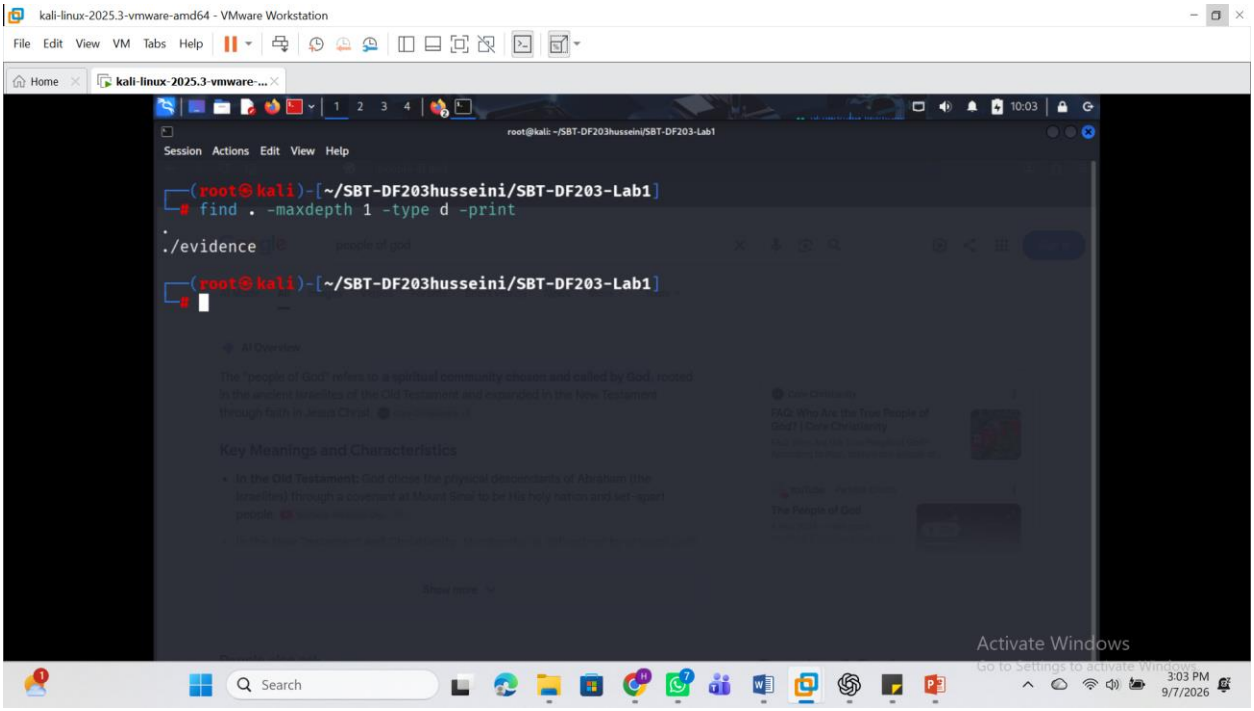
Item	Recommended Tool	Purpose
Operating system	Kali Linux or Ubuntu VM	Run Apache, curl, tshark and Wireshark.
Web server	Apache2	Serve the training webpage locally.
Capture tools	Wireshark and tshark	Capture and inspect network packets.
Client	curl and a web browser	Generate controlled HTTP requests.
Hashing	sha256sum	Verify capture integrity.
Optional second host	Windows or Linux VM	Produce Ethernet frames with visible MAC addresses.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab1
pwd
find . -maxdepth 1 -type d –print
```



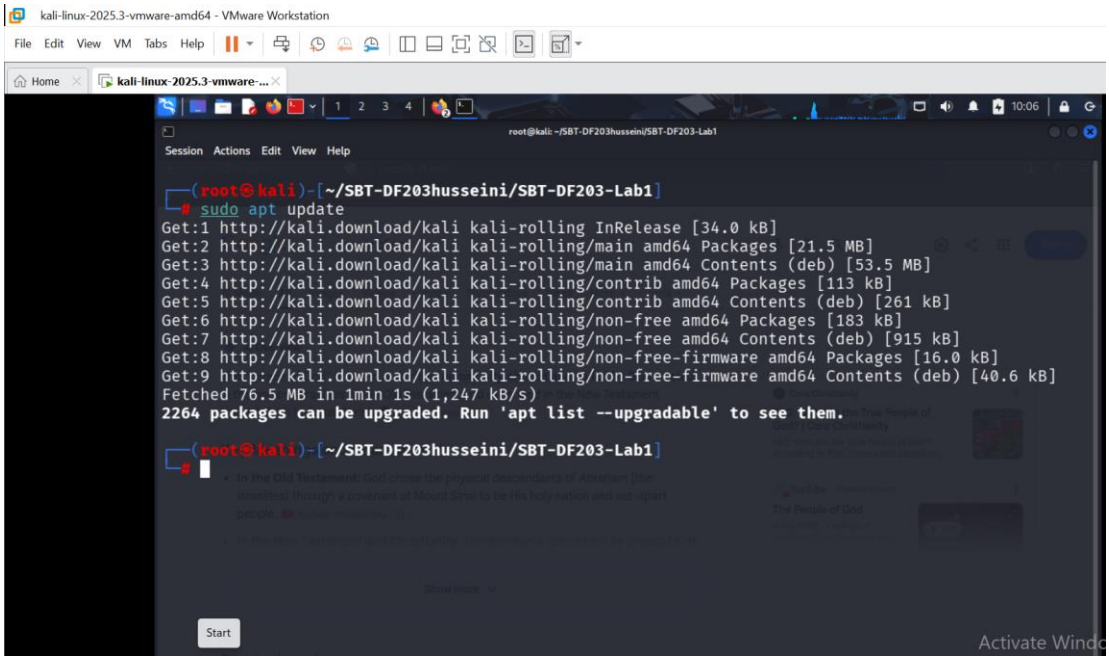
Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.



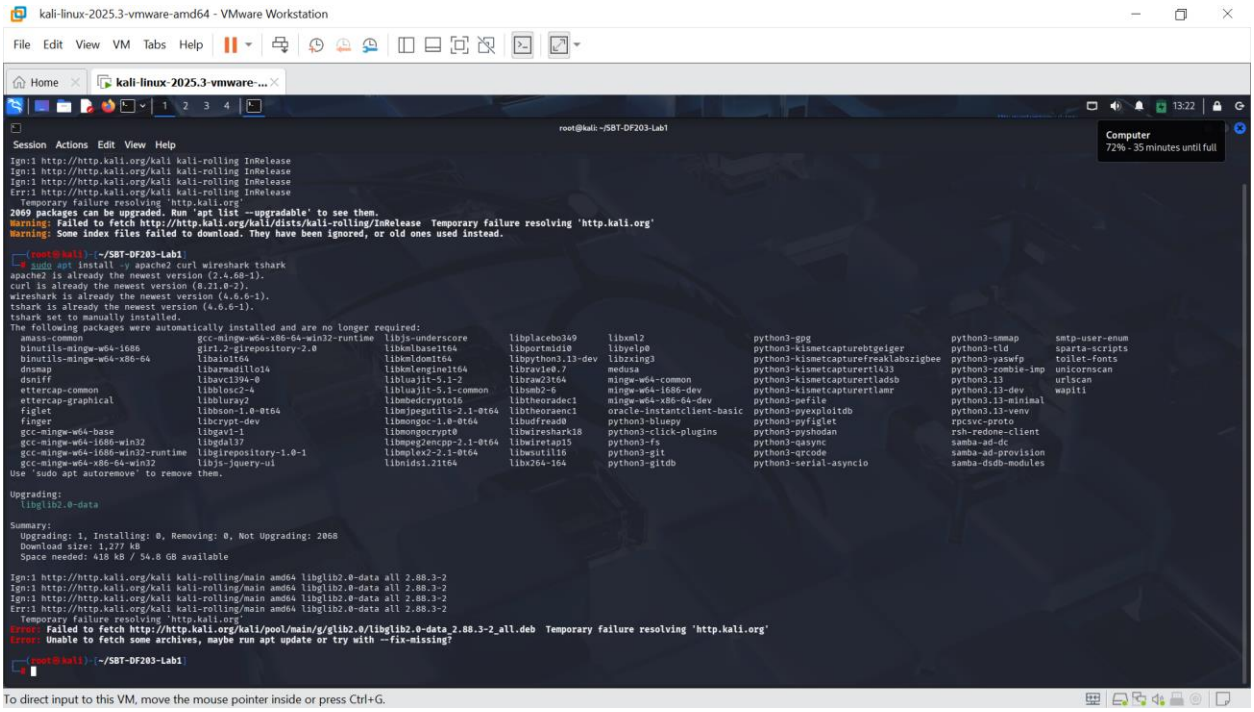

```
sudo apt update
sudo apt install -y apache2 curl wireshark tshark
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: YOUR FULL NAME</p><p>Reg
No: YOUR REGISTRATION NUMBER</p></body></html>\n' | sudo tee /var/www/html/basic.html
curl -v http://127.0.0.1/basic.html
# After capture:
sha256sum evidence/basic.pcapng | tee reports/basic_pcap_sha256.txt
```

Screenshot required: Apache status, the completed webpage in a browser, folder structure and the SHA-256 of the capture.

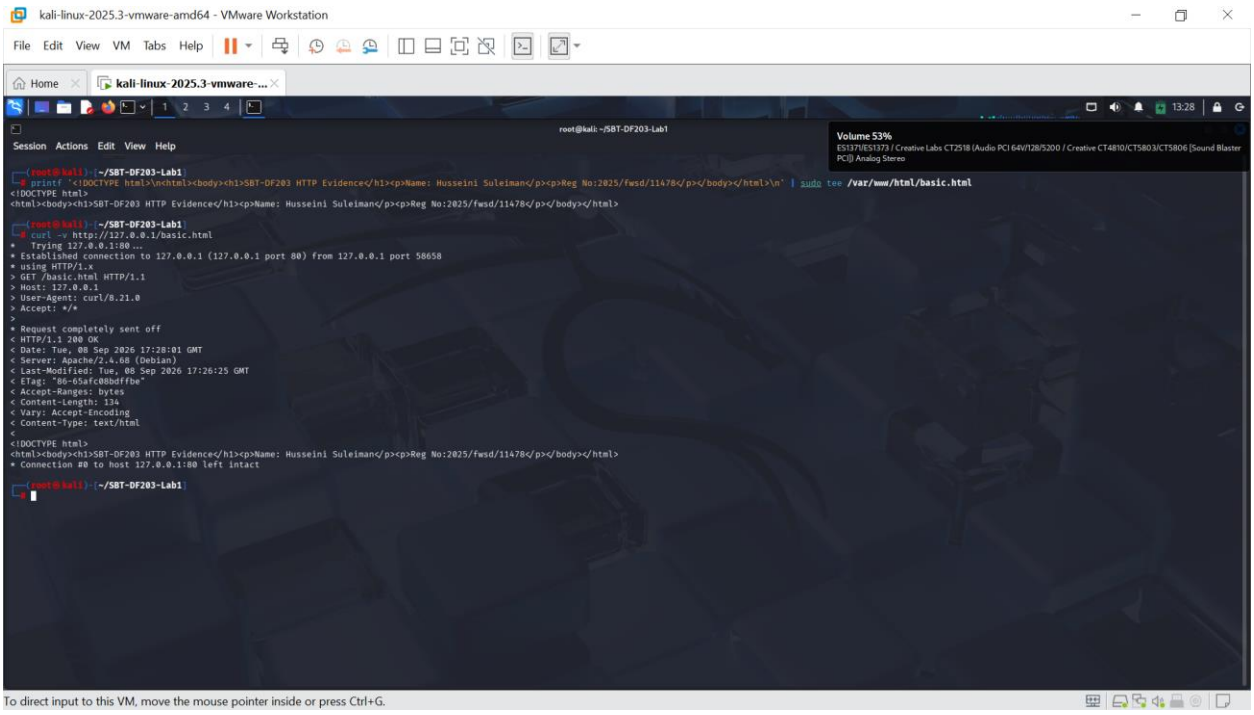
Answers: using the command; `sudo apt update`



See answer while using the command: `sudo apt install -y apache2 curl wireshark tshark`



Using the command: `printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: YOUR FULL NAME</p><p>Reg No: YOUR REGISTRATION NUMBER</p></body></html>\n' | sudo tee /var/www/html/basic.html`



7. Mini Evidence and Chain-of-Custody Worksheet

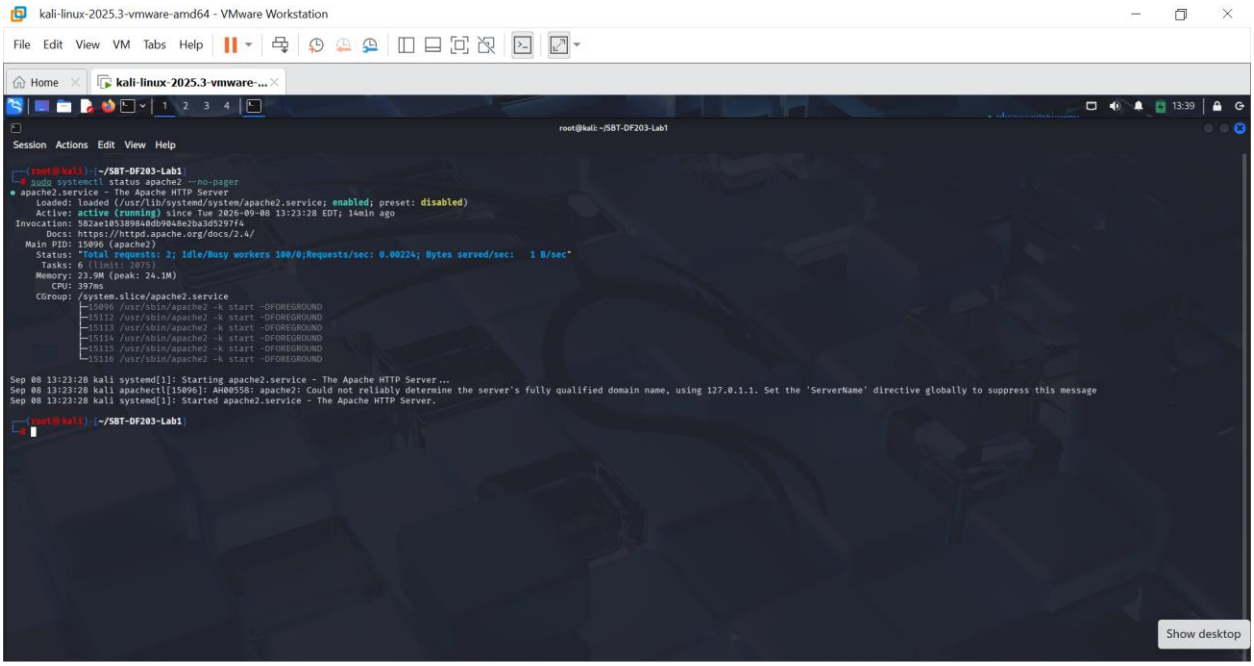
Field	Student Entry
Case/lab identifier	SBT-DF203husseini/SBT-DF203-Lab1
Trainee name	Husseini Suleiman
Date and time started	
Evidence file name(s)	
Source or generation method	
Original SHA-256	
Working-copy SHA-256	
Analysis workstation/VM	
Notes on any changes	

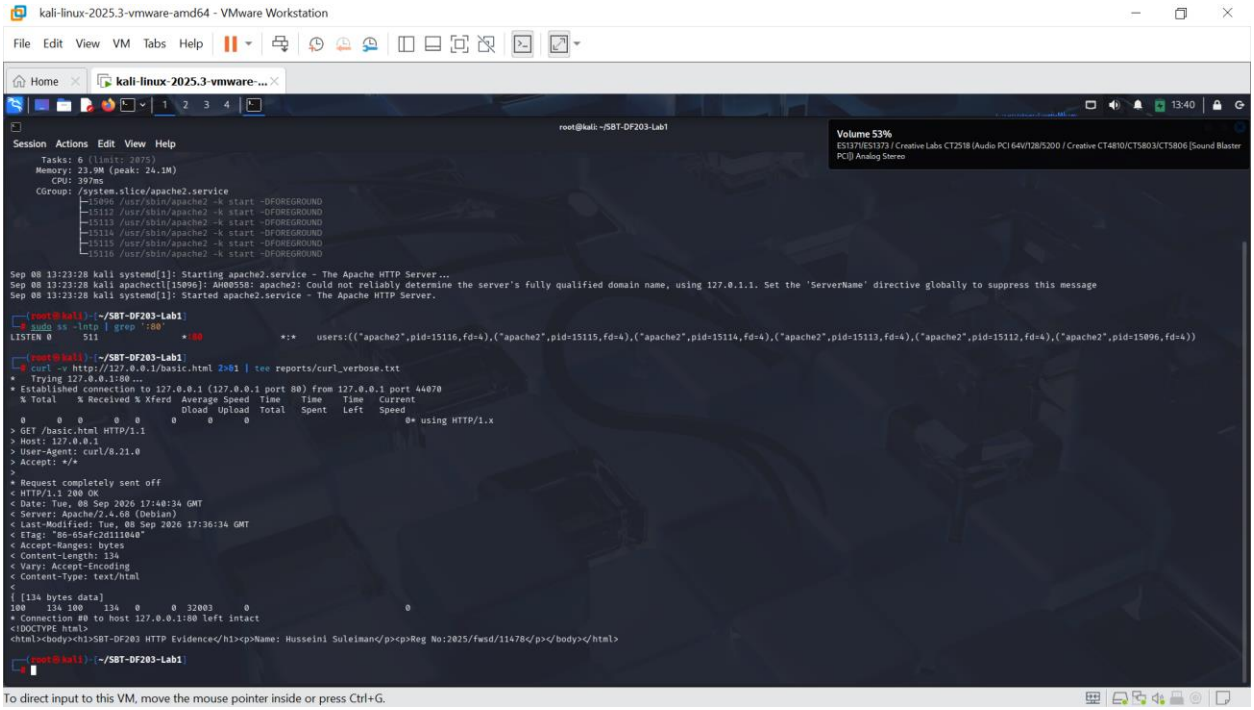
8. Part A - Verify the Web Service and Generate Text HTTP Traffic

1. Replace the name and registration number placeholders in basic.html with your own details.
2. Open http://127.0.0.1/basic.html and confirm that the page displays correctly.
3. Run curl with verbose output and save the request/response headers.
4. Record the local service state and listening port.

```
sudo systemctl status apache2 --no-pager
sudo ss -lntp | grep ':80'
curl -v http://127.0.0.1/basic.html 2>&1 | tee reports/curl_verbose.txt
```

USING THE ABOVE COMMAND: See evidence below





9. Part B - Capture the Session

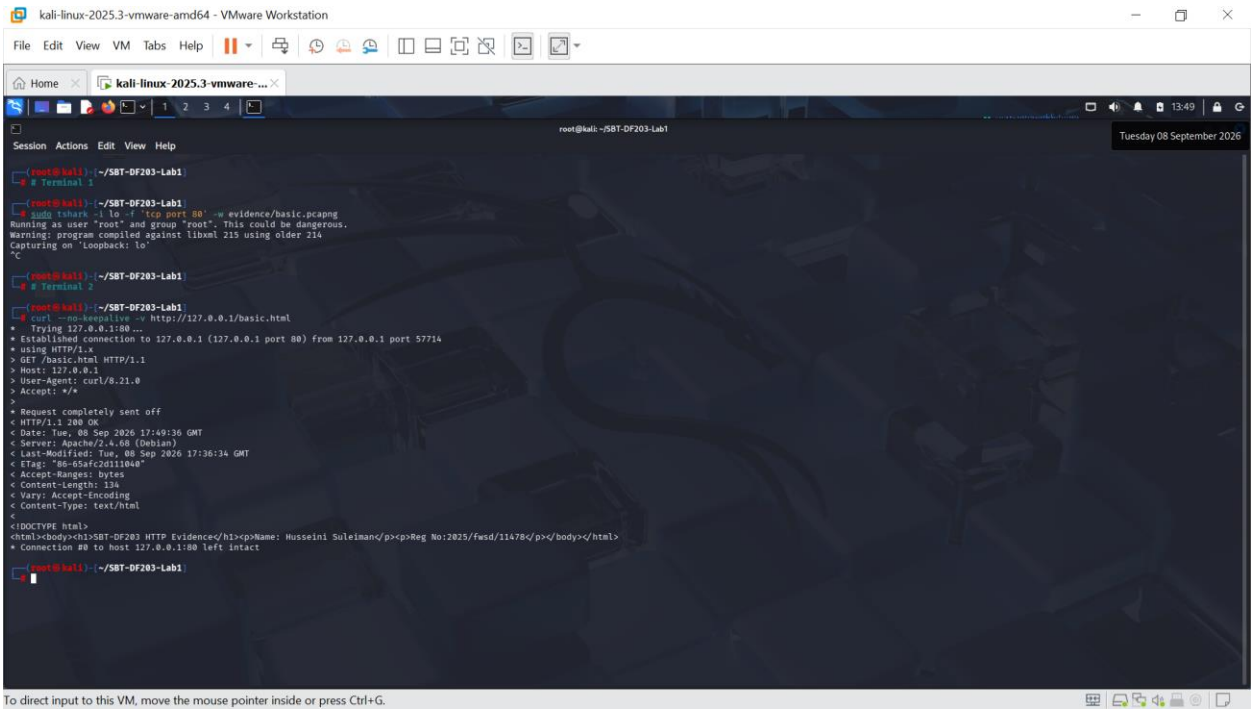
Use either Wireshark or tshark. The loopback interface is normally lo. Start the capture before running curl and stop it after the page is received.

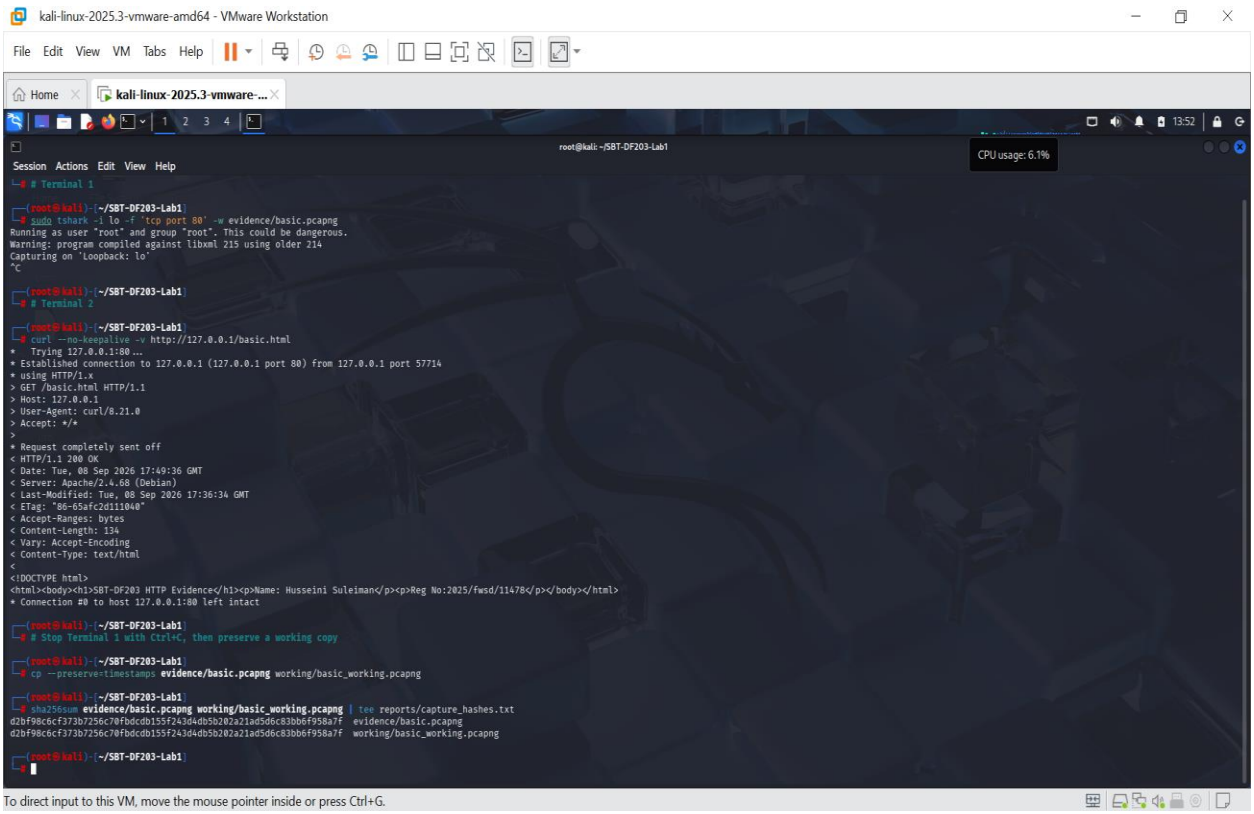
```
# Terminal 1
sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng

# Terminal 2
curl --no-keepalive -v http://127.0.0.1/basic.html

# Stop Terminal 1 with Ctrl+C, then preserve a working copy
cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng
sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt
```

Wireshark Alternative
Open Wireshark, select lo, apply capture filter tcp port 80, start capture, run curl, stop capture, and save as evidence/basic.pcapng.

See screenshot for the below command

```
# Terminal 1
sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
```





Hash Value

d2bf98c6cf373b7256c70fbdcdb155f243d4db5b202a21ad5d6c83bb6f958a7f evidence/basic.pcapng

d2bf98c6cf373b7256c70fbdcdb155f243d4db5b202a21ad5d6c83bb6f958a7f working/basic_working.pcapng

10. Part C - Analyze the TCP Three-Way Handshake

Open the working copy and apply the display filter tcp.stream eq 0. Identify the first three packets and complete the table.

Packet	Flags	Client Seq	Server Seq/ACK	Timestamp	Interpretation
1	SYN		Trying 127.0.0.1:80...	Date: Tue, 08 Sep 2026 17:49:36 GMT	http://127.0.0.1/basic.html
2	SYN, ACK	using HTTP/1.x			Server acknowledges and provides its initial sequence number.
3	ACK				Client acknowledges; the connection becomes established.

```
tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 && tcp.flags.syn==1' \
  -T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.flags -e tcp.seq -e tcp.ack | tee reports/handshake.tsv
```

11. Part D - Examine the HTTP Request and Response

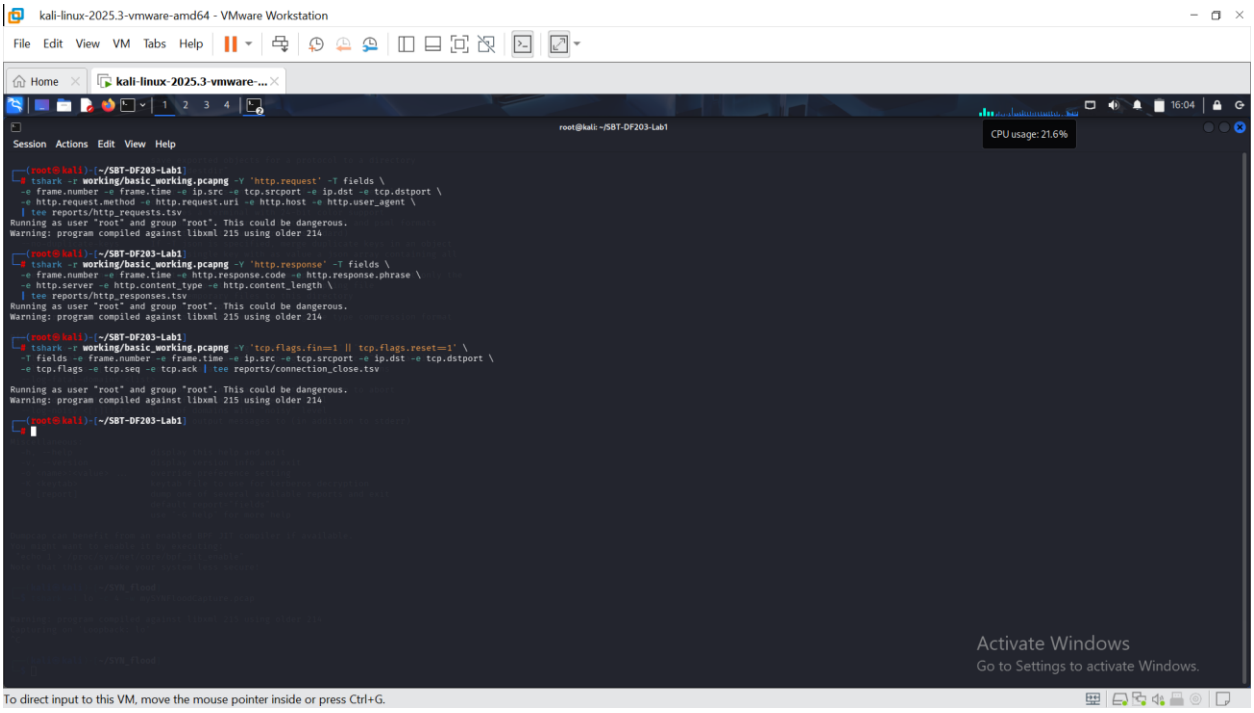
5. Filter for http.request and identify the GET method, request URI, Host and User-Agent.
6. Filter for http.response and record the status code, response phrase, server and content type.
7. Use Follow TCP Stream to reconstruct the complete request and response.
8. Compare the TCP payload length with the sequence/acknowledgement progression.

```
tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host -e http.user_agent \
  | tee reports/http_requests.tsv

tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e http.response.code -e http.response.phrase \
  -e http.server -e http.content_type -e http.content_length \
  | tee reports/http_responses.tsv
```

12. Part E - Inspect Encapsulation and Connection Closure

9. Select the HTTP GET frame and expand Frame, Internet Protocol, Transmission Control Protocol and Hypertext Transfer Protocol.
10. Explain the relationship among frame length, IP total length, TCP header length and TCP payload length.
11. Locate FIN/ACK packets or a reset that closes the session. Explain the observed termination sequence.
12. State why a loopback capture does not provide normal source/destination MAC addresses. For MAC evidence, repeat with two authorized VMs on the same host-only network.

```
tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
  -T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
```



clear

13. Required Forensic Findings

Finding	Value/Observation
Capture start and end time	
Client IP and port	
Server IP and port	
Client initial sequence number	
Server initial sequence number	
HTTP request timestamp	
Requested URI	
HTTP response code	
Content type and length	
Connection termination method	
MAC-address observation	

Required Screenshots Checklist

☐ Webpage showing your name and registration number.
☐ Apache service listening on TCP port 80.

☐ Capture running on the selected interface.
☐ Three-way handshake with sequence and acknowledgement numbers.
☐ HTTP GET request and complete headers.
☐ HTTP 200 response and HTML content.
☐ Follow TCP Stream output.
☐ TCP connection closure.
☐ Capture hashes.
☐ Optional two-VM capture showing source and destination MAC addresses.

Student Report Template

13. Cover page and executive summary.
14. Authority, scope and evidence description.
15. Environment and acquisition method.
16. Chronological packet timeline.
17. Handshake analysis.
18. HTTP request/response reconstruction.
19. Encapsulation and MAC-address explanation.
20. Findings, limitations and conclusion.
21. Appendix of commands, hashes and screenshots.

Submission Requirements

- One PDF report named SBT-DF203-Lab1_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Preparation and evidence handling	10	Correct webpage, folders, capture preservation and hashes.
Traffic capture	15	Valid capture containing a complete HTTP session.
TCP handshake analysis	20	Correct flags, ports, timestamps, sequence and acknowledgement interpretation.
HTTP analysis	25	Correct request, response, headers, payload and stream reconstruction.
Encapsulation and closure	15	Correct layer and termination analysis, including loopback/MAC explanation.
Report quality	15	Clear timeline, screenshots, references and conclusions.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
Default server port	TCP 80.
Handshake	SYN, SYN-ACK, ACK.
Request	GET /basic.html with Host and User-Agent headers.
Response	Usually HTTP/1.1 200 OK and text/html.
Loopback MACs	Normal Ethernet MAC addresses are not present on lo.
Sequence rule	ACK normally points to the next byte expected; SYN consumes one sequence number.

Command Reference Summary

Command	Purpose
curl -v URL	Generate an HTTP request and show headers.
tshark -i lo -w file.pcapng	Capture loopback traffic.
http.request	Wireshark display filter for HTTP requests.
tcp.stream eq 0	Display one TCP conversation.
Follow TCP Stream	Reconstruct application data.
sha256sum file	Create an integrity hash.

Academic Integrity and Authorization Statement

Student Declaration
I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.